AcadMate Technical Paper

AcadMate

Academic Advising and Degree Planning

F2026-Team3

CS 691 Sprint 2

7 November 2026

Contents

- 1 Executive Summary
- 2 Project Context and Objectives
- 3 Users and Use Cases
- 4 Requirements and User Stories
- 5 System Architecture
- 6 Data Model
- 7 Functional Design
- 8 AI and Analytics Design
- 9 Implementation
- 10 Testing and Quality
- 11 Sprint 2 Work and Results
- 12 Deployment and Operations
- 13 Limitations and Risks
- 14 Conclusion and Next Steps
- References

1 Executive Summary

AcadMate is a web application for students, faculty advisors and administrators. Students can review academic progress and prepare a future term plan. Advisors can find an advisee, inspect the academic record and approve or return the plan. Administrators maintain accounts, course data and institution settings.

The application uses React [1] and Vite for the frontend, a FastAPI REST API [2], and SQLAlchemy [3] with PostgreSQL [4] for persistence. In the latest local regression, 353 main application backend tests passed. Browser checks passed the student and advisor workflows, including an approval that remained visible after the student refreshed the plan. The advisor name search also passed seven focused checks after its input binding was repaired.

Academic risk prediction remains incomplete. A separate feature branch contains data preparation, training, calibration and inference code, but the two original frozen model artifacts are absent. Its API therefore returns a service unavailable response for model inference. The application leaves the probability unscored. SHAP explanations and retrieval from approved university documents still have no implementation evidence in the reviewed snapshots.

2 Project Context and Objectives

Problem

Academic advising depends on information spread across student records, course catalogs, grades, attendance, assignments, calendars and conversations. Students may not know what they still need, advisors may spend time assembling context manually, and administrators need a reliable view of accounts, advisor capacity and system activity. AcadMate brings these workflows into one platform and supports AI assistance with configured data and topic controls.

Objectives

- Give students a clear view of progress, courses, deadlines and degree requirements.
- Let students submit future term plans for advisor approval and receive timely notices.
- Give advisors and administrators role appropriate views of students, approvals, workload and audit activity.
- Use academic data to support risk detection and recommendations while retaining human review for sensitive topics.
- Provide a maintainable platform with documented APIs, tests, seed data and deployment instructions.

Scope

The main application contains authentication, student pages, advisor pages, administration and supporting services. Password login and role controls were tested locally. Google sign in, external mail delivery, calendar synchronization and live Claude responses were not accepted through external provider tests. The academic risk research branch is separate from main and has not been merged.

3 Users and Use Cases

Table 1. User roles and representative workflows

User	Primary needs	Representative flows
Student	Understand academic standing and plan next steps	Dashboard, courses and grades, degree plan, AI advisor, schedule, assignments, messages, meetings and privacy settings
Advisor	Review advisees, approve plans and communicate	Advisor dashboard, student history, approvals, meetings, messages and workload context
Administrator	Maintain platform data, policies and access	Overview, audit log, users, advisors, catalog drafts, integrations, AI settings and reports



Figure 1. Student persona asset

4 Requirements and User Stories

The condensed product backlog contains 13 stories and 110 estimated points. It covers authentication, the student experience and administration. These are product backlog estimates. The 110 points do not establish Sprint 1 velocity or the Sprint 2 commitment. Advisor work and risk research are also described in a later ten task plan.

Table 2. Condensed user story backlog

Epic	Stories	Points	Scope
E1 Authentication and Access	2	13	Sign in, sessions, navigation, account setup and recovery
E2 Student Experience	7	55	Dashboard, degree plan, AI advisor, schedule, assignments, messages, meetings and privacy
E3 Administration	4	42	Overview, audit log, user and advisor management, catalog, integrations and institution AI settings
Total	13	110	Condensed feature backlog

Acceptance criteria examples

- A student who enters an incorrect password five times is rate limited and the failed attempt is audited.
- A student cannot add a course when a prerequisite is missing and receives a specific explanation.
- A submitted degree plan becomes pending approval for the advisor and creates a notice.
- An AI question about a blocked topic is handed to a human meeting flow without a model call.
- Two students cannot reserve the same meeting slot concurrently; one request receives a conflict response.

5 System Architecture

The browser calls the FastAPI API through the Vite development proxy or a configured API URL. FastAPI checks the session and role before services access PostgreSQL. Student plan submission creates an advisor approval request. The advisor decision changes the stored plan status, which the student sees on refresh.

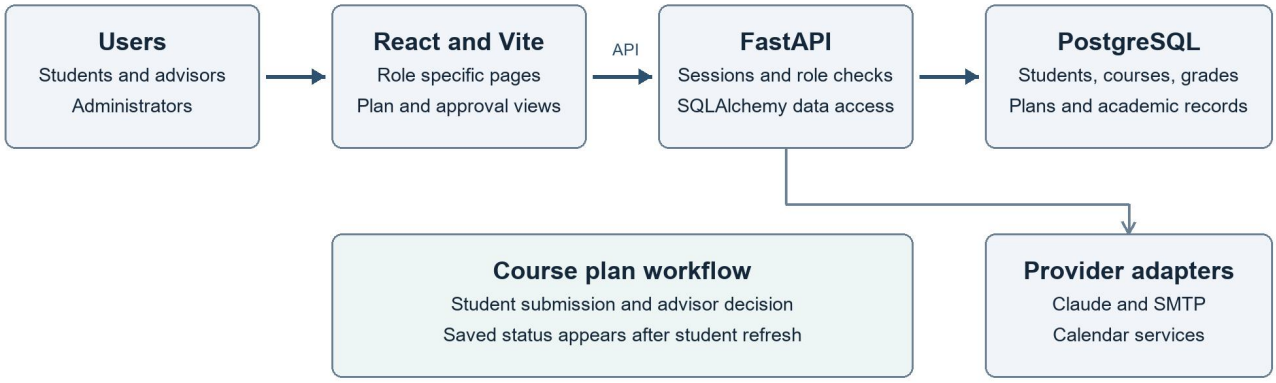


Figure 2. Main application architecture and course plan workflow.

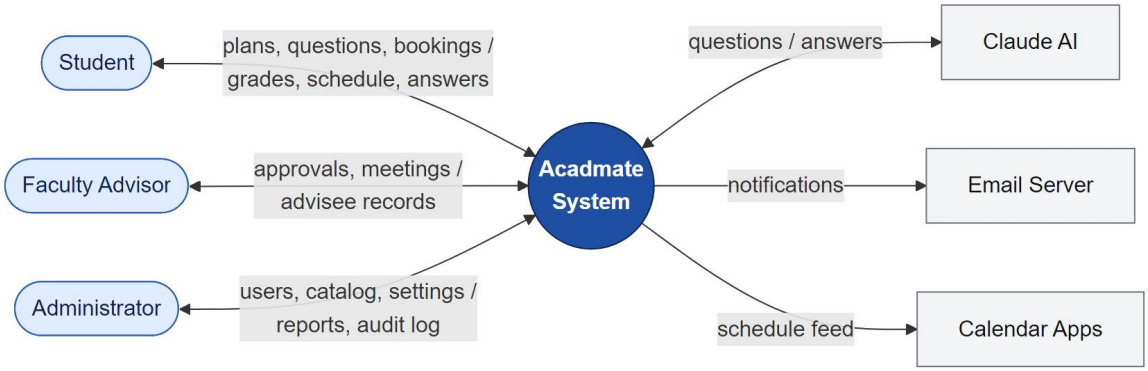


Figure 3. AcadMate context diagram

Service boundaries

- Frontend: React routes and reusable UI components for student, advisor and administrator workflows.
- API: FastAPI routers for authentication, users, students, courses, AI, calendar, assignments, conversations, staff, meetings, administration and approvals.

- Persistence: SQLAlchemy models and PostgreSQL tables for users, students, advisors, courses, terms, enrollments, grades, plans, meetings, messages, AI threads, flags, audit events and integrations.
- External service adapters exist for Claude, email and calendar workflows. The architecture diagram shows intended boundaries. Local tests do not establish that these providers are configured or working in an institutional environment.

6 Data Model

The data model separates identity, academic records, advising workflows and AI conversations. Users may have a student or advisor record. Students connect to programs, terms, enrollments, grades, assignments, plans, meetings, messages and AI threads. Advisors connect to advisees, approvals and meetings. Administrative actions are represented by audit events, while integration jobs and API keys support controlled data exchange.

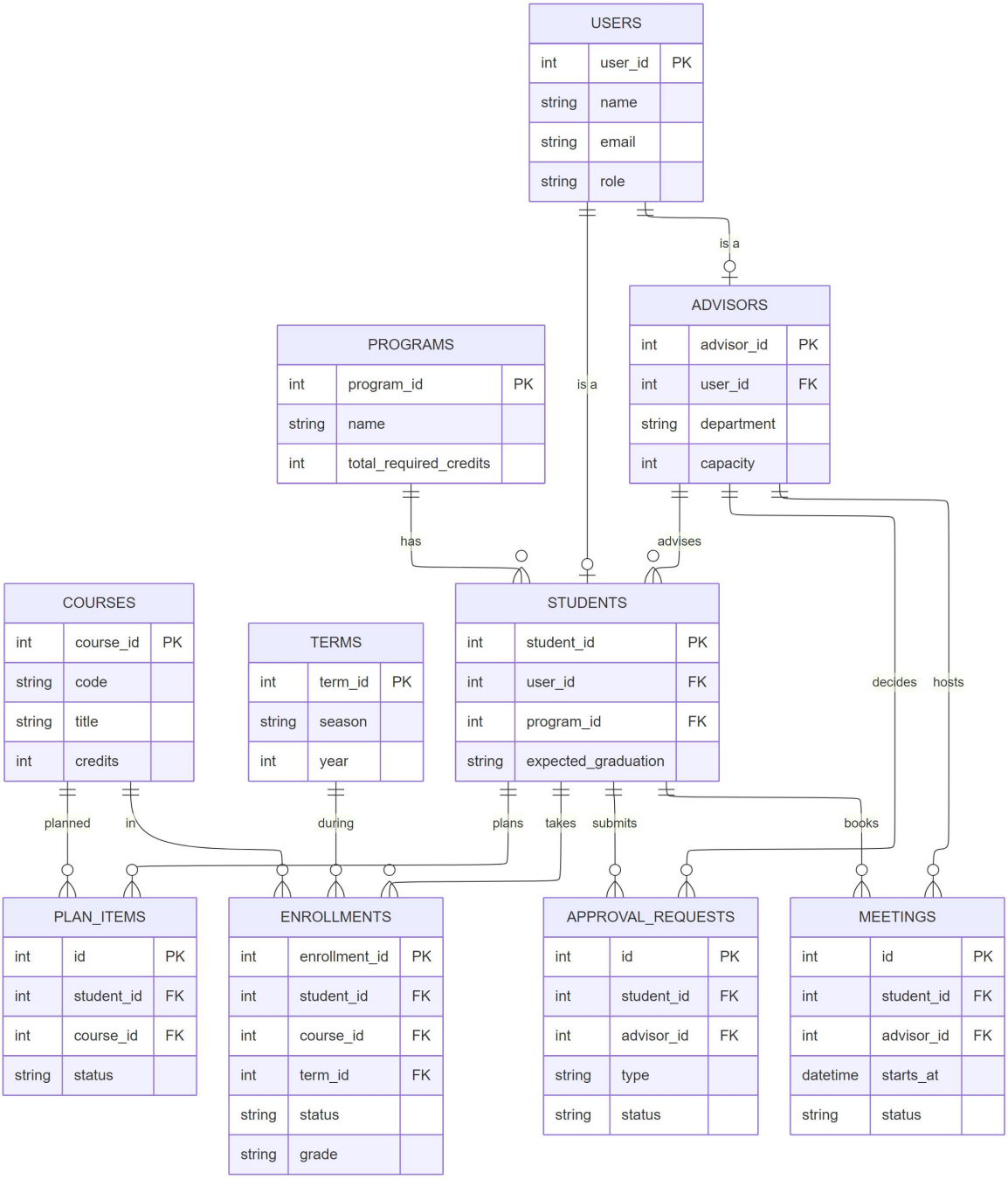


Figure 4. Simplified entity relationship diagram

The current repository has no migration system. A schema change requires rebuilding tables and reseeding demo data, which is suitable for a controlled demo but creates operational risk for production data. A migration strategy should be added before deployment to a persistent environment.

7 Functional Design

Authentication and access

Authentication uses session cookies and role checks. The backend includes password login, invite and reset tokens, deactivation, idle session handling and failed attempt limits. The browser regression used

separate student and advisor sessions and confirmed that a student could not open protected advisor content. Demo credentials are synthetic and must not be used for a live institution.

Student experience

The student dashboard shows GPA, credit totals and current academic context. For the seeded student Amara Okafor, regression checks confirmed a GPA of 3.73, 21 earned credits out of 36 required, nine credits in progress and six credits remaining after the current term. The degree plan shows requirement groups and future course rows. Spring 2027 contains CS 680 and CS 691 in the demonstrated approval flow.

Advisor and administrator workflows

Advisor pages include the dashboard, advisee list, student record, approvals, meetings, messages, reports and settings. The browser regression confirmed eight advisees and five pending requests for Dr. Smith. A local fix connects the advisee search field to the page search state, so entering Amara reduces the list to one matching student. Approval request 1709 changed both planned course rows to Approved, and the student saw that result after refreshing. Reopen restored the demo plan to Pending approval.

Communication and notifications

AcadMate keeps advising messages, notices, meeting changes and approval events inside the application. Email is used for account invites and selected notifications. The repository documents that password reset emails are not sent by the demo environment; reset links are printed to the server log.

8 AI and Analytics Design

The main AI advisor uses permitted academic database context and conversation history. Topic guards can hand a sensitive question to a human meeting flow. The service supports Anthropic Claude when configured and a rule based engine when Claude is unavailable. Tests cover application behavior with controlled inputs. No live Claude response, approved document retrieval or measured improvement in advising quality is claimed.

Risk research branch

- The branch origin/feature/academic-risk-prediction contains the OULAD feature pipeline [5], a 16-feature logistic regression baseline, sigmoid probability calibration [6], evaluation code and prediction routes.
- An application demonstration uses 100 synthetic students, 800 enrollments and 19,200 rows across eight extension tables. These records support integration exercises and do not measure accuracy on real students.
- The frozen logistic_regression.joblib and calibration_maps.joblib files are absent, preventing local reproduction of the original training and inference results.
- The research cutoff of 0.5 is diagnostic. An advisor outreach threshold requires a defined policy and model evaluation. SHAP assigns feature contributions to individual predictions [7]. The current engagement and grade summaries are descriptive and do not implement SHAP.

On 7 November, 41 ML unit tests passed. The research backend had 15 passes, three failures and one error when the real artifact smoke test was enabled. Missing model files blocked the four inference checks. The frontend had seven passes and one skipped real model check. The research build passed, but these results do not establish working model inference.

9 Implementation

Table 3. Implementation stack

Layer	Technology	Repository evidence
Frontend	React 19, React Router, Vite, Tailwind	frontend/ and student, advisor and admin pages
Backend	Python 3.10+, FastAPI, Pydantic, SQLAlchemy	backend/main.py, routers, schemas and services
Database	PostgreSQL	backend/models.py, database.py, seed_data.py
AI	Anthropic Claude with rule based fallback	backend/services/ai and student/admin AI routers
Testing	pytest and representative workbook test pack	backend/tests and Acadmate_Test_Cases_15.xlsx
Development	Docker, GitHub, GitHub Wiki	README.md and Doumentation/

The main API exposes versioned routes under /api/v1 and API documentation at /docs. The frontend uses VITE_API_URL and API_PROXY_TARGET. Static prototype files remain design references. The regression checked main commit 0eda245a7b05 with the local advisor search change. The risk branch snapshot was ff472bf961aa. The source package records the exact files and the local change separately.

10 Testing and Quality

Main backend tests use pytest [8]. The historical 15-case workbook is separate. Counts refer to checks, not completed features or story points.

Table 4. Regression results on 7 November 2026

Suite	Result	Coverage or limitation
Main backend	353 passed	No failures, errors or skips
Browser workflow	12 passed	Core flows, research unavailability and runtime checks
Advisor search	7 passed	Name match, combined filters, no match and clearing
Risk ML unit tests	41 passed	Feature and pipeline logic
Risk backend	15 passed 3 failed 1 error	Real model inference blocked by absent artifacts
Risk frontend	7 passed 1 skipped	Optional real model check skipped

Both frontend builds passed. The main build retains a chunk size warning. Backend tests recreate and seed an isolated test database. The application date is fixed at 1 October 2026 for repeatability. Browser checks use an isolated demonstration database, and approval request 1709 was restored after the run. The research cutoff of 8 February 2027 is a simulation input. These checks do not establish accessibility, load capacity, external provider delivery, model accuracy or production security.

11 Sprint 2 Work and Results

The repository contains two Sprint 2 records with different scopes. A historical task log lists three completed issues for 26 points. A later planning workbook lists ten subtasks without point estimates and retains Not Started in its original snapshot. Current code and test evidence are mapped to all ten tasks without rewriting the original planning record.

Table 5. Current evidence for the four planned workstreams

Workstream	Current evidence	Open acceptance work
Advisor profile work	Password login, advisor pages and the core approval flow passed local checks	External provider integration tests
Degree progress	Requirement groups and credit totals verified for the seeded student	Institutional degree policy validation
Academic risk	Data, training and inference code on a separate branch	Original frozen files, reproduced inference and outreach policy
AI foundations	Database context and rule based assistance in main	SHAP and approved document ingestion and retrieval

Table 6. Sprint 2 task log snapshot

Issue	Work	Points	Task log status
SCRUM-19	Backend APIs	8	Done
SCRUM-20	Student Dashboard	9	Done
SCRUM-21	Advisor Dashboard	9	Done
Total	Sprint 2 task-log snapshot	26	Task-log total

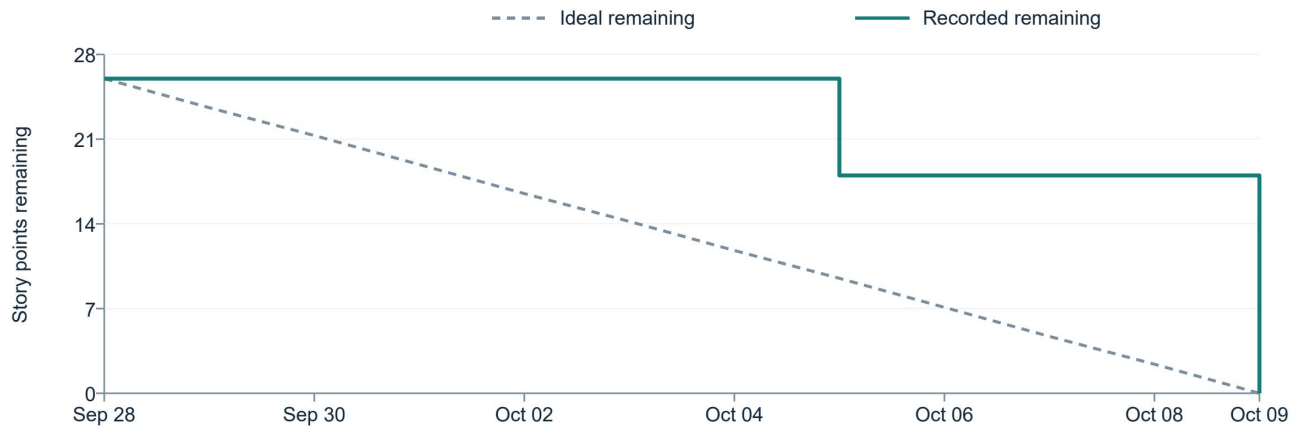


Figure 5. Historical three issue Sprint 2 burndown, 28 September to 9 October 2026.

The historical log records eight points completed on 5 October 2026 and 18 on 9 October. The standalone chart covers 28 September to 9 October and reaches zero for those three issues. The same log records 24 points for Sprint 1, giving a two-sprint average of 25, excluding Sprint 0. Its Sprint 2 completed-to-committed ratio is 26/26. These metrics describe only the three-issue task log. The later ten-task planning workbook has no point estimates, so the same ratio does not apply to it.

12 Deployment and Operations

The repository documents Python, Node.js, Docker and PostgreSQL for local development. The installation manual in this package provides PowerShell commands, separate development and test database settings and expected health responses. In the existing regression environment, the main frontend uses port 55173 and the API uses 58000. These local validation ports are separate from the README defaults of 5173 and 8000.

Local setup sequence

1. Start PostgreSQL in Docker on port 5433 and set DATABASE_URL in backend/.env.
2. Install backend dependencies, rebuild the demo schema and load seed data.
3. Start the FastAPI server and confirm /api/v1/health returns status ok.
4. Install frontend dependencies, set VITE_API_URL and API_PROXY_TARGET, then start Vite.
5. Run the backend test suite against the test database before demonstrating changes.

Schema reset deletes the selected database contents. It belongs only in a disposable demonstration or test database. Production installation needs a reviewed schema migration, HTTPS, restricted database access, managed secrets, backups and a restore exercise. This package documents a proposed deployment sequence. No production installation or institutional security acceptance has been performed.

13 Limitations and Risks

- Model inference is blocked by two missing frozen artifacts. Risk research remains on a separate branch. SHAP and approved document retrieval are unfinished.
- There is no database migration system. Rebuilding the schema deletes existing data.
- The application uses synthetic demo data. External services, accessibility, performance under load and institutional deployment have not passed a production acceptance process.

14 Conclusion and Next Steps

AcadMate supports the advising workflow from student degree audit to advisor decision and student refresh. The main application passed the local regression. Risk inference remains blocked by missing model artifacts. The three-issue task log and the ten-task plan describe different scopes and cannot be combined into one completion percentage.

References

[1] Meta. Quick Start. React documentation. Accessed November 9, 2026. https://react.dev/learn

[2] FastAPI Contributors. FastAPI documentation. Accessed November 9, 2026. https://fastapi.tiangolo.com/

[3] SQLAlchemy Authors. SQLAlchemy ORM. SQLAlchemy 2.1 Documentation. Accessed November 9, 2026. https://docs.sqlalchemy.org/en/21/orm/

[4] PostgreSQL Global Development Group. PostgreSQL Documentation. Accessed November 9, 2026. https://www.postgresql.org/docs/

[5] J. Kuzilek, M. Hlosta, and Z. Zdrahal. Open University Learning Analytics dataset. Scientific Data, vol. 4, article 170171, 2017. DOI: 10.1038/sdata.2017.171. https://doi.org/10.1038/sdata.2017.171

[6] scikit-learn developers. Probability calibration. scikit-learn documentation. Accessed November 9, 2026. https://scikit-learn.org/stable/modules/calibration.html

[7] S. M. Lundberg and S.-I. Lee. A Unified Approach to Interpreting Model Predictions. arXiv:1705.07874, 2017. https://arxiv.org/abs/1705.07874

[8] pytest-dev team. pytest documentation. Accessed November 9, 2026. https://docs.pytest.org/en/stable/